

Sorting

Kanishk Goel

9 May, 2026

1 Counting Sort

non comparison based stable sorting algorithm. Good when the range of elements are $< n \log n$.

Solution
$A \leftarrow$ initialise input array $m \leftarrow \max(A)$ $freq \leftarrow m + 1$ length array with counts of every element i at $freq[i]$ Compute prefix sum $ans \leftarrow$ initialise answer array for $i \leftarrow n - 1$ to 0 do $ans[freq[i] - 1] \leftarrow A[i]$ $freq[i] = freq[i] - 1$

Runtime - $O(n)$, is not an in-place sorting algorithm and requires $O(n)$ space as well.

2 Radix Sort

Runs a stable sorting algorithm for individual units of each element from right to left. Works only on the assumption of an upperbound on length of elements.

Runtime - $O(d \times (b + n))$ where d is the number of digits, b is the base of number system used. Space required is $O(n + b)$

3 Bucket Sort

Create n empty buckets, each bucket is a linked list. Sort each bucket using insertion sort and then combine all buckets.

- Worst Case - $O(n^2)$ (when one bucket gets all the elements)
- Best Case - $O(n + k)$ (when every bucket gets equal number of elements). Assumes k is linearly proportional to n .

- Space - $O(n + k)$